

Total Marks: 03

Obtained Marks:

Data Structure and Algorithms

Assignment # 01

Last Date of Submission: 2nd Oct 2026

Submitted To: Ms. Sobia Rasheed Malik

Student Name: Anique Ali

Registration Number: 25108113



Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Instructions: Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.

Question 1 — Data Structures and ADT

a) Differentiate between static and dynamic data structures. Give two examples of each and explain one advantage and one limitation of each type.

b) Consider a List ADT that supports the following operations:

- insert(position, value)
- delete(position)
- search(value)
- get(position)
- isEmpty()

Explain why the ADT concept separates the logical behavior of a list from its implementation. Briefly describe how the same List ADT could be implemented using: (1) an array and (2) a singly linked list.

Answer 1:

a) Static vs. dynamic data structures

A static data structure has a fixed size that is decided before the program runs (at compile time), and its memory is reserved in one block. A dynamic data structure can grow and shrink while the program is running, because memory is requested and released at run time (usually from the heap).

Point	Static data structure	Dynamic data structure
Size	Fixed once declared	Changes at run time
Memory allocation	Compile time, contiguous block	Run time, often scattered in memory
Examples	Array, fixed-size stack/queue built on an array	Linked list, dynamic stack/queue, tree
Advantage	Fast direct access by index in $O(1)$; simple, no pointer overhead	Uses memory only as needed; easy to insert/delete without shifting
Limitation	Memory is wasted if the array is too big, and it overflows if it is too small	Needs extra memory for pointers, and access is slower (no direct indexing)

For example, if we declare `int marks[100]` but only 30 students exist, 70 slots are wasted. If 120 students come, the array cannot hold them. A linked list avoids this problem because a node is created only when a student is actually added.

b) List ADT: logical behaviour vs. implementation

An Abstract Data Type (ADT) describes what a data structure does (its operations and their expected results) and says nothing about how it is done internally. For the List ADT, a user only needs to know that `insert(position, value)`, `delete(position)`, `search(value)`, `get(position)` and `isEmpty()` exist and what each one returns. This separation is useful because:

The programmer can use the list without knowing how it is stored (information hiding).

The implementation can be changed later (for example array to linked list) without changing the code that uses the list.

It makes programs easier to design, test and maintain, since each part depends only on the interface.

Implementation using an array: The elements are kept in consecutive memory locations together with a variable size that stores the current number of elements. `get(position)` is simply `arr[position-1]`. `insert()` shifts the elements after the position one step to the right and then places the value. `delete()` shifts the later elements one step to the left. `search()` scans the array from the start. `isEmpty()` checks whether `size == 0`. The capacity is fixed, so insertion fails when the array is full.

Implementation using a singly linked list: Every element is stored in a node containing the data and a pointer to the next node, and a head pointer points to the first node. `insert()` creates a new node and adjusts the next pointers after walking to the required position. `delete()` changes the next pointer of the previous node so the target node is skipped, and then frees it. `get()` and `search()` traverse the nodes one by one from the head. `isEmpty()` checks whether `head == NULL`. There is no fixed capacity, but there is no direct indexing either.

Question 2 — Sequential List Using Array

A sequential list is implemented using an array of capacity N . The following operations are performed:

- Insert at beginning
- Insert at end
- Insert at position k
- Delete from beginning
- Delete from end
- Delete from position k
- Search for an element

- For each operation, determine its worst-case time complexity using Big O notation.
- Explain why inserting an element at the beginning of an array-based list can require more operations than inserting an element at the end.
- Suppose the list contains: 10 20 30 40 50. Show the array after inserting 25 at position 3, assuming positions start from 1.

a) Worst-case time complexity of each operation

Let n be the number of elements currently stored in the list ($n \leq N$).

Operation	Worst case	Reason
Insert at beginning	$O(n)$	All n elements must shift right by one place
Insert at end	$O(1)$	No shifting, just put the value at index n
Insert at position k	$O(n)$	Up to n elements shift (worst case $k = 1$)
Delete from beginning	$O(n)$	The remaining $n-1$ elements shift left
Delete from end	$O(1)$	Just reduce the size by one

Delete from position k	$O(n)$	Elements after k shift left (worst case $k = 1$)
Search for an element	$O(n)$	Linear search may check every element

Note: search is $O(n)$ because the list is not assumed to be sorted. If it were sorted, binary search could reduce it to $O(\log n)$.

b) Why inserting at the beginning costs more than inserting at the end

In an array, elements sit in consecutive locations and every element has a fixed index. To insert at the beginning, index 0 must be empty, so every existing element has to be moved one position to the right first. With n elements that means n shifts, so the cost grows with the size of the list, which is $O(n)$. When inserting at the end, the next free slot (index n) is already empty, so the value is placed directly without moving anything. That takes a constant number of steps, which is $O(1)$. So the position of the insertion decides how many elements must move.

c) Inserting 25 at position 3 (positions start from 1)

The original list is: 10 20 30 40 50. Position 3 currently holds 30, so 30, 40 and 50 are shifted one place to the right and 25 is placed at position 3.

Before:

Position	1	2	3	4	5	6
Value	10	20	30	40	50	-

After:

Position	1	2	3	4	5	6
Value	10	20	25	30	40	50

Final array: **10 20 25 30 40 50**.

Complexity Analysis of Algorithms

Determine the time complexity in Big O notation of each algorithm. Show your reasoning. a)

```
for(int i = 0; i < n; i++)
{
    cout << i;
}
```

b)

```
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
```

```
cout << i << j;
}
```

c)

```
for(int i = 0; i < n; i++)
{
    cout << i;
}

for(int j = 0; j < n; j++)
{
    cout << j;
}
```

d)

```
int i = 1;

while(i < n)
{
    cout << i;
    i = i * 2;
}
```

e) Consider an algorithm that uses an array of n elements and creates another array containing n elements. Determine its time complexity and additional space complexity.

a)

```
for(int i = 0; i < n; i++)
{    cout << i;    }
```

The loop runs from $i = 0$ to $n-1$, so the statement inside runs n times, and each run takes constant time. Total = $c \times n$. **Time complexity: $O(n)$.**

b)

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        cout << i << j;
```

The outer loop runs n times. For each value of i , the inner loop also runs n times. So the print statement runs $n \times n = n^2$ times. **Time complexity: $O(n^2)$.**

c)

```
for(int i = 0; i < n; i++)    cout << i;
for(int j = 0; j < n; j++)    cout << j;
```

These are two separate loops, one after the other (not nested). The first takes n steps and the second takes n steps, so the total is $n + n = 2n$. In Big O we drop constants, so **time complexity: $O(n)$.**

d)

```
int i = 1;
while(i < n)
{    cout << i;    i = i * 2;    }
```

The value of i doubles every time: 1, 2, 4, 8, 16, ... After k iterations, $i = 2^k$. The loop stops when $2^k \geq n$, that is when $k = \log_2(n)$. So the loop runs about $\log_2(n)$ times. For example, if $n = 16$ it runs for $i = 1, 2, 4, 8$, which is $4 = \log_2(16)$ times. **Time complexity: $O(\log n)$.**

e)

The algorithm reads the n elements of the given array and copies or creates n elements in a new array. It touches each element a constant number of times, so the time complexity is $O(n)$. The new array needs room for n elements, in addition to the input, so the additional (auxiliary) space complexity is $O(n)$.

Question 4 — Linked Lists and Complexity Comparison

Consider the following three list implementations:

- (1) Singly Linked List
- (2) Doubly Linked List
- (3) Circular Linked List

a) For each type, briefly explain how nodes are connected.

a) How the nodes are connected

(1) Singly linked list: Each node has two parts: data and one pointer (next). The next pointer holds the address of the following node. The last node's next is NULL. We can move only in the forward direction, starting from the head.

(2) Doubly linked list: Each node has three parts: a prev pointer, the data, and a next pointer. The next pointer points to the following node and the prev pointer points to the previous node. The prev of the first node and the next of the last node are NULL. We can move both forward and backward.

(3) Circular linked list: It is like a singly linked list, but the next pointer of the last node points back to the first node instead of NULL. So the nodes form a closed loop and there is no NULL at the end. (A doubly circular version also exists, where the last node's next points to the first and the first node's prev points to the last.)

b) Complete the following table by giving the typical worst-case time complexity for each operation.

Assume a tail pointer is not maintained unless explicitly stated.

Operation	Singly Linked List	Doubly Linked List	Circular Linked List
Insert at beginning	$O(1)$	$O(1)$	$O(n)^*$
Insert at end*	$O(n)$	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$	$O(n)$
Delete first node	$O(1)$	$O(1)$	$O(n)^*$
Delete a node after a given node	$O(1)$	$O(1)$	$O(1)$

* For insert at end, assume no tail pointer is maintained unless otherwise specified.

*For a circular list that keeps only a head pointer, adding or deleting the first node requires walking to the last node so that its next pointer can be updated to the new first node, which takes $O(n)$. If the list keeps a pointer to the last (tail) node instead, both of these become $O(1)$, and insert at end also becomes $O(1)$. This is the reason circular lists are often stored with a tail pointer. Since the question says to assume no tail pointer, the values above are given for the head-only case.

For insert at end in the singly and doubly lists, we must traverse from the head to reach the last node, hence $O(n)$. For search, in every list the worst case is that the element is at the end or not present, so all n nodes are checked. For deleting a node after a given node, we already hold a pointer to the given node, so only a few pointers change, which takes constant time.

c) A student claims: "A doubly linked list is always faster than a singly linked list because it has two links." Analyze this statement. Explain why having an additional pointer does not automatically make every operation faster.

c) "A doubly linked list is always faster than a singly linked list"

This statement is **not correct**. Having two links helps in some operations, but not all of them:

Where it is faster: Deleting a node when we already have a pointer to that node is $O(1)$ in a doubly linked list (the node knows its previous node), whereas in a singly linked list we have to traverse from the head to find the previous node, which is $O(n)$. Traversing backwards is also possible only in the doubly list. If a tail pointer exists, deleting the last node is $O(1)$ in a doubly linked list but $O(n)$ in a singly one.

Where it is not faster: Search is $O(n)$ in both lists, since we still go node by node. Insert at the beginning, delete the first node, and insert or delete after a given node are all $O(1)$ in both, so there is no gain in Big O terms.

It can even be slower in practice: Every insertion or deletion in a doubly linked list has to update two pointers (prev and next) instead of one, so the constant work per operation is higher and there is more room for pointer mistakes. It also uses more memory.

So an extra pointer gives more flexibility (backward movement, easy deletion of a known node), but it does not change the complexity of operations such as searching, and it adds cost. The right choice depends on which operations the application performs most often.

d) For a list containing n elements, explain the additional memory requirement of a doubly linked list compared with a singly linked list.

d) Additional memory required by a doubly linked list

Each node of a doubly linked list stores one extra pointer (prev) compared with a singly linked list. For a list of n nodes, this means **n extra pointers**. If one pointer needs p bytes, the extra memory is $n \times p$ bytes. For example, on a 64-bit system $p = 8$ bytes, so a list of 1000 nodes

needs $1000 \times 8 = 8000$ bytes (about 8 KB) more; on a 32-bit system $p = 4$ bytes, so it would be 4000 bytes.

In terms of growth, the extra memory is proportional to n , so it is $O(n)$ additional space overall, or a constant amount per node. This is the price paid for being able to traverse in both directions.

DSA BS(AI)-3A SZABIST-ISB